Day 59 – API Management (APIM) – Part 8: Key Value Maps & Extension Policies

Module: SAP API Management – Key Value Map Operations (Mediation), Extension Policies

---

Checked against SAP's documentation – your content is accurate; three small precision points (marked in the text):
1) **Message Validation** validates an **XML** message against an **XSD** schema (it is not a generic schema validator for JSON).
2) In the Assign Message demo, variables are referenced with **curly braces**, i.e. **{northwindResponse.content}** – your handwritten version looks like "@…#", so just make sure the braces are what you type in the policy.
3) For an Open Connector type API, **only one** Open Connectors policy can be attached.

1. Key Value Map (KVM) Policy – what is it? (start here first)

Key Value Map Operations is the last of the **Mediation policies.** A **Key Value Map** stores **key** → **value pairs** and lets you do **value mapping** – i.e. **input in, output out** (give a key, get the mapped value).

Comparison with CPI: in CPI you do this with the **Value Mapping artifact**. In APIM there is no such artifact – instead you create a **Key Value Map** and read it through the **Key Value Map Operations** policy.

Step 1 – create the Key Value Map

```
Menu -> Configure -> APIs -> Key Value Maps -> Create
```

Practical example: KVM name **kvm_value_mapping_demo**

Key	Value
India	IN
Australia	AUS

Step 2 – use it from an API Proxy

API Proxy → **Policy section** → **Key Value Map Operations (KVM).** The policy documentation shows the supported operations:

- **Put** – writes/updates an entry in the KVM.
- **Delete** – deletes an entry.
- **Get** – reads a value (the retrieved value is assigned to a **variable** via **assignTo**).

Demos were done in the system for Put entry, Get operation and Delete operation. Example for Get: the value is stored into a variable (e.g. **sapapim.countrycode**) using the key **India**:

```
Get -> assignTo variable: sapapim.countrycode India (result: IN)
```

*Extra: a KVM can also be created as **encrypted**; values of an encrypted map are read into a variable whose name starts with **private.** – useful for storing credentials.*

These are the ways we can use a KVM. With this, the **Mediation policies topic is closed.**

2. Now, Extension Policies

- **1.** JavaScript
- **2.** Message Logging
- **3.** Message Validation
- **4.** Open Connectors
- **5.** Python Script
- **6.** Service Callout
- **7.** Statistics Collector

*(Written in your notes as "Statistic Collector" – the official name is **Statistics Collector.**)*

3. Service Callout – the main important policy here

Service Callout is like **Request-Reply in CPI** – if you want to **call an API** (an external service) from inside the proxy flow, you use the Service Callout policy.

Scenario: Source → data → **in between, an API call** → for this we use the Service Callout policy.

The Service Callout pattern uses three steps (three policies):

Step	Policy	Role
1	Assign Message	Builds the **request message** that is sent to the external service (headers, query params, verb, payload).
2	Service Callout	References that message, defines the **target URL**, and gives a **name to the response object** returned by the target service.
3	Extract Variables	Parses the callout response using **JSONPath / XPath** and sets variables from it.

Assign Message can also be used **afterwards** – when you want to **generate (send) data into the Response** that goes back to the caller, e.g. set the response payload from the stored callout response.

Demo – Northwind OData

- Service Callout policy calls the public **Northwind OData** service (**/Customers**). In the default policy content, the **SSL Info** section was removed for the demo.
- The callout's response is given the name **northwindResponse** – i.e. the data is being **stored** in a variable, **not passed on as the request**.
- **Debug** was used to check how the Service Callout policy behaved.
- Then an **Assign Message** policy with **Set Payload** returns that stored response: **{northwindResponse.content}** (curly braces – see correction box).

```
Scenario flow: Service Callout (northwind /Customers) -> Assign Message (Set
Payload) -> response to client
```

4. The other Extension policies

Policy	What it does	Scenario from class

Message Validation	Validates an **XML** message against an **XSD schema** (validation based on the schema).	Reject XML that doesn't match the schema.
Message Logging	Sends syslog-style log messages to a **third-party log management service**.	Send APIM message logs to **Splunk, Sumo Logic, Loggly**.
Open Connectors	Attached to an **Open Connector** type API (only one such policy per API) – used when you need to call Open Connectors.	Calling a 3rd-party app through Open Connectors.
JavaScript	Attach **custom JavaScript** code to a proxy flow.	Custom logic not available in standard policies.
Python Script	Attach **custom Python** code to a proxy flow.	Same idea, in Python.
Statistics Collector	Collects statistics from message data (e.g. product ID, price, user ID) and sends them to analytics for reports.	Generate a **custom dashboard/report on User ID**; shows the statistics report.

Quick-recall summary

- **KVM** = APIM's value-mapping store (CPI's Value Mapping equivalent). Created under **Configure** → **APIs** → **Key Value Maps**; used via **Key Value Map Operations** with **Put / Get / Delete**.
- Mediation policies are now complete: Access Entity, Assign Message, Extract Variables, JSON to XML, Key Value Map Operations, Raise Fault, XSL Transform.
- **Extension policies (7):** JavaScript, Message Logging, Message Validation, Open Connectors, Python Script, Service Callout, Statistics Collector.
- **Service Callout** = APIM's Request-Reply; always paired with **Assign Message** (build request) and **Extract Variables** (parse response with JSONPath/XPath).
- **Message Validation** = XML vs XSD; **Message Logging** = Splunk/Sumo Logic/Loggly; **Statistics Collector** = custom analytics.